

Data Visualization Guide

Solavise Women in Data (SWID) Programme • Matplotlib & Seaborn

A chart is only useful if it shows the right thing. Always ask: **What am I trying to show?** Then pick the chart that answers that question best.

1 Setting Up: Always Start Here

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
# Set a clean visual style
sns.set_theme(style='whitegrid')
plt.figure(figsize=(8, 5))
```

Seaborn vs Matplotlib: Matplotlib gives you full control. Seaborn makes things look polished with less code. Use Seaborn for most charts and Matplotlib to fine-tune the details.

2 Which Chart Should I Use?

What you want to show	Best chart type	Library
Compare categories (e.g. sales by region)	Bar Chart	Seaborn / Matplotlib
Show change over time (e.g. monthly revenue)	Line Chart	Seaborn / Matplotlib
See how two numbers relate (e.g. age vs income)	Scatter Plot	Seaborn
Show the spread of one number (e.g. exam scores)	Histogram	Seaborn / Matplotlib
Compare distributions across groups	Box Plot	Seaborn
Show how values are distributed smoothly	KDE Plot	Seaborn
Show correlation between all number columns	Heatmap	Seaborn
Show parts of a whole (e.g. market share)	Pie Chart	Matplotlib

3 Chart Code: Ready to Use

Bar Chart

Use when: comparing values across categories

```
sns.barplot(data=df, x="Category", y="Sales")
plt.title("Sales by Category")
plt.show()
```

Line Chart

Use when: showing trends over time

```
sns.lineplot(data=df, x="Month", y="Revenue")
plt.title("Monthly Revenue")
plt.show()
```

Scatter Plot

Use when: exploring the relationship between two numbers

```
sns.scatterplot(data=df, x="Age", y="Income",
hue="Gender")
plt.title("Age vs Income")
plt.show()
```

Histogram

Use when: seeing how one number is distributed

```
sns.histplot(data=df, x="Score", bins=20,
kde=True)
plt.title("Score Distribution")
plt.show()
```

Box Plot

Use when: comparing spread and spotting outliers across groups

```
sns.boxplot(data=df, x="Department",
y="Salary")
plt.title("Salary by Department")
plt.show()
```

Heatmap (Correlation)

Use when: finding which columns relate to each other

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="Blues",
fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

4 Making Your Chart Look Good

Add titles and axis labels

```
plt.title("My Chart Title")
plt.xlabel("X Axis Label")
plt.ylabel("Y Axis Label")
```

Show a legend

```
plt.legend(title="Group")
# or remove legend:
plt.legend().remove()
```

Change the figure size

```
# Call this BEFORE your chart code
plt.figure(figsize=(10, 6))
# width=10 inches, height=6 inches
```

Rotate x-axis labels

```
# Helpful when labels overlap
plt.xticks(rotation=45)
plt.tight_layout() # fix cut-offs
```

Choose a colour palette

```
sns.set_palette('Set2') # soft colours
sns.set_palette('Blues') # single colour range
sns.set_palette('husl') # bright and distinct
```

Save your chart

```
# Save BEFORE plt.show()
plt.savefig('my_chart.png', dpi=150,
bbox_inches='tight')
plt.show()
```

Show multiple charts side by side

```
fig, axes = plt.subplots(1, 2,
figsize=(12, 5))
sns.barplot(data=df, x="Cat", y="Val",
ax=axes[0])
sns.lineplot(data=df, x="Date", y="Val",
ax=axes[1])
plt.tight_layout()
plt.show()
```

5 Quick Tips for Clean Visualizations

Always label your axes. A chart without labels is hard to understand for anyone reading it.

Keep titles short and clear. A good title tells the reader exactly what the chart is about.

Do not use too many colours. Stick to 2 to 4 colours max. More colours make charts harder to read.

Use `plt.tight_layout()`. This stops your labels from getting cut off at the edges.

Call `plt.show()` at the end. This displays the chart. Without it, nothing will appear.

Save before you show. `plt.savefig()` must come before `plt.show()` or the file will be blank.